

DSCI 523 Quiz 2

Tidy control flow: groups, missingness, values

```
# Change selected values; preserve all other values
fixed <- fix_me |>
  mutate(province = case_when(
    province == "Alberta" ~ "AB",
    province == "British Columbia" ~ "BC",
    TRUE ~ province
  ))

# Drop rows using only selected columns, or using every
column
penguins |> drop_na(body_mass_g)
penguins |> drop_na()

# Check missingness; never compare with == NA
is.na(x)

• case_when(condition ~ value, ..., TRUE ~ old_col): rowwise
branches; without the final default, unmatched values become NA. Results
must have compatible types. For a factor column, as.character() before
text replacement.
• drop_na(x:y): remove rows missing in any listed column only.
drop_na(): remove rows missing anywhere. Decide whether to drop,
replace, or preserve missingness.

# One row per group
planes |>
  group_by(engine) |>
  summarise(
    avg_seats = mean(seats, na.rm = TRUE),
    planes = n()
  )

# Keep original rows; calculate within each group
gapminder |>
  group_by(country) |>
  mutate(life_exp_gain = lifeExp - first(lifeExp)) |>
  ungroup()
```

• group_by(a, b): each (a, b) combination is a group; it marks data but
does not collapse rows. summarise() gives one row/group; grouped
mutate() usually keeps the original rows. n() counts the current group.
• mean()/max()/median() etc. propagate NA unless na.rm = TRUE is
passed inside that function. Use ungroup() before later global operations.
• Order: desired output observation → group keys → summarise() or
grouped mutate() → missing-value rule → select()/arrange() →
check shape.

purrr::map*: suffix = output shape

<pre>library(purrr) map(mtcars, median) # list map_dbl(mtcars, median, na.rm = TRUE) # double vector map_lgl(mixed_bag, is.numeric) # logical vector map_chr(x, function(item) paste0("id-", item)) map_df(mtcars, median) # tibble (classroom convention)</pre>		
Desired result from every .x element	Use	Check
arbitrary/variable	map(.x, .f, .. list .)	
one double	map_dbl	length-1 numeric
one integer	map_int	length-1 integer
TRUE/FALSE	map_lgl	length-1 logical
one string	map_chr	length-1 character
combined tibble	map_df	results combine as a tibble

- .f can be median, is.numeric, or function(item) ...; ... passes extra arguments to every call, e.g. na.rm = TRUE.
- Before mapping: identify .x, per-element output type/length, suffix, and missing-value arguments. map() is still a list even when displayed elements are numeric.
- For needs manual preallocation, iteration extent and index management. If asked for map*, use map*, not across().

Functions, returns, lazy evaluation, scope

```
mpg_to_kml <- function(mpg) {
  if (!is.numeric(mpg)) {
    stop("mpg must be numeric")
  }
  mpg * 0.425144 # last expression is
returned
}

repeat_string <- function(x, n = 2) {
  out <- ""
  for (i in seq_len(n)) out <- paste0(out, x)
  out
}

• Definition: name <- function(parameters) { body }. Functions are
objects and can be passed to map.
```

- Last expression returns; return(value) exits early. A default is used only when its argument is omitted.
- Lazy evaluation: an argument is evaluated only when the body accesses it. ... accepts extra arguments; Lecture 6 marks manual ... handling Optional-Advanced.
- Scope: inner names mask outer names; missing names are found in enclosing/global/package environments at call time (dynamic lookup). Each call gets a fresh execution environment. Prefer explicit parameters to mutable globals.
- pkg::fun() calls one package function without attaching the package. source("src/functions.R") loads functions from a script.

Tests, TDD, errors

<pre>library(testthat) test_that("conversion works", { expect_equal(mpg_to_kml(1), 0.425144) expect_equal(mpg_to_kml(2), 0.850288) }) test_that("invalid input errors", { expect_error(mpg_to_kml("A")) expect_error(mpg_to_kml(list(1, 2))) expect_error(mpg_to_kml(data.frame(x = 1))) })</pre>	
Assert	Expectation
exact value/type/ attributes	expect_identical(x, y)
equal or near-equal values	expect_equal(x, y, tolerance = ...)
equality without attribute emphasis	expect_equivalent(x, y)
logical result	expect_true(x) / expect_false(x)
error / warning / printed output	expect_error(expr) / expect_warning(expr) / expect_output(expr, ...)

- TDD: write normal, edge and error specifications → implement the smallest function → add cases and rerun. A failed test is evidence about the contract or implementation; do not delete it blindly.
- Check bad inputs at function entry and stop("useful message"); warning() allows continuation. try({ ... }) continues after an error but does not make bad input valid.
- Use a meaningful numeric tolerance only when the contract allows floating-point error.

Documentation, variance, and final checks

```
#' Convert miles per gallon to kilometres per litre.
#'
#' @param mpg numeric vector in miles per gallon
#' @return numeric vector in kilometres per litre
#' @examples
#' mpg_to_kml(1)
mpg_to_kml <- function(mpg) mpg * 0.425144
```

• roxygen2: description + @param + @return + @examples (Worksheet 6 also shows @export). Document the contract, not every obvious line.

- Lab 3 sample variance (write from scratch, not var()): sum((x - mean(x))^2) / (length(x) - 1). Return one numeric value; reject list/data-frame inputs with a tested error.
- Function review: explicit inputs/outputs/errors; one coherent task; readable names/indentation; no accidental global dependency or repeated work.

1. "Each group" → group keys + summarise; "retain rows" → grouped mutate.
2. Missing value → choose drop_na(columns), drop_na(), na.rm = TRUE, or case_when replacement.
3. case_when → include TRUE ~ old; check factor/character type and use is.na() for missingness.
4. map → suffix equals required output; pass na.rm through
5. Function → parameters/defaults → input check → return shape → clear error message.
6. Test → expectation matches contract; include a hand-checkable edge/error; tolerance only with reason.
7. Before submitting code, check parentheses, commas, names, arguments, grouping state and final type/shape.